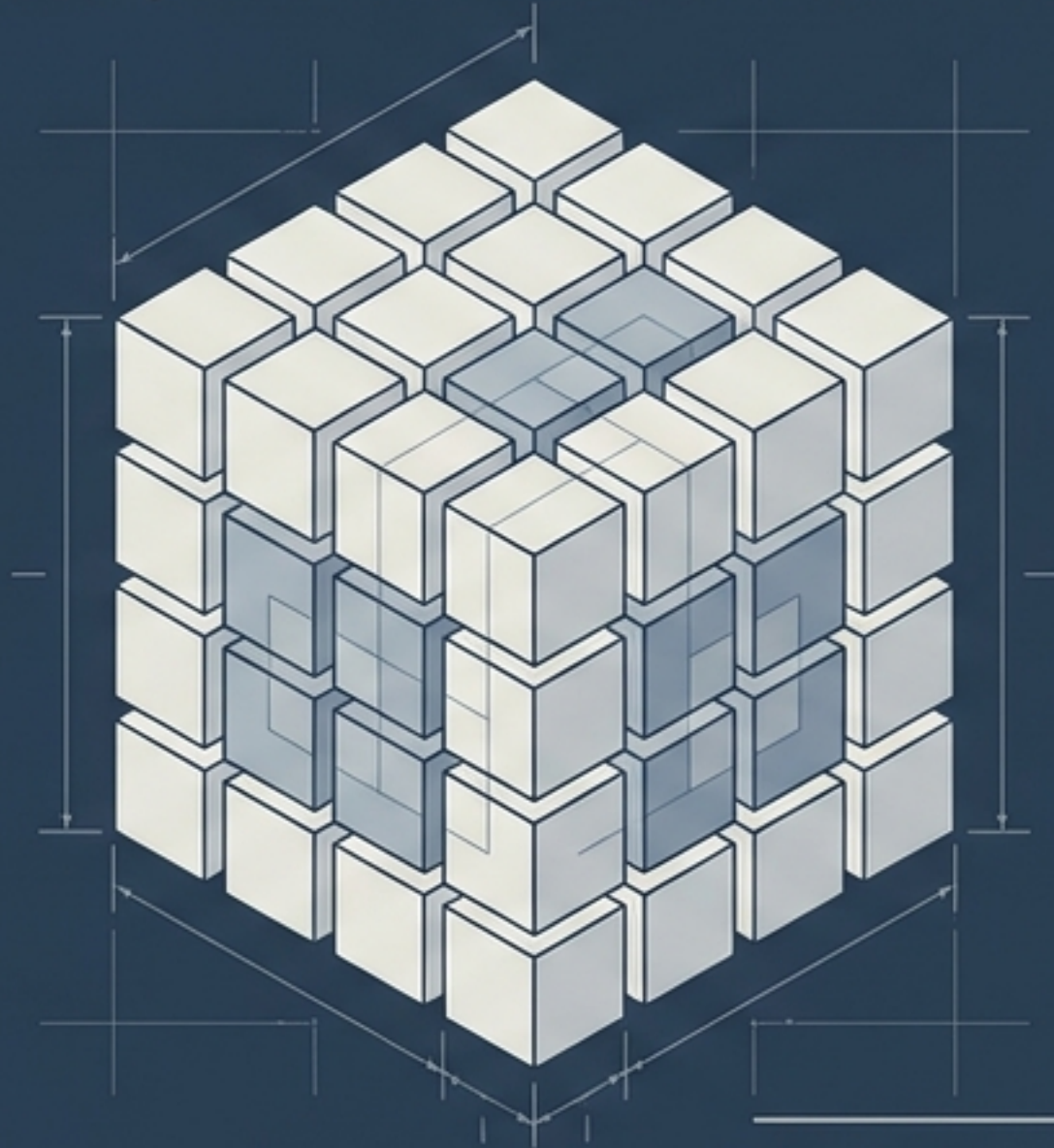


Arquitetura de API: O Padrão Monolito Modular

Estruturando Aplicações Backend Escaláveis e Manuteníveis



Disciplina: Projeto de Software – Prof. Gleison Brito

O Paradigma: Unificação Externa, Separação Interna



Um único sistema



Um único deploy



Um único banco de dados



Módulos internos bem separados

O Monolito Modular extrai o melhor dos dois mundos: a simplicidade operacional de um monolito com a organização de código de microsserviços.

A Separação Ocorre no Domínio de Negócio

Módulo Usuários

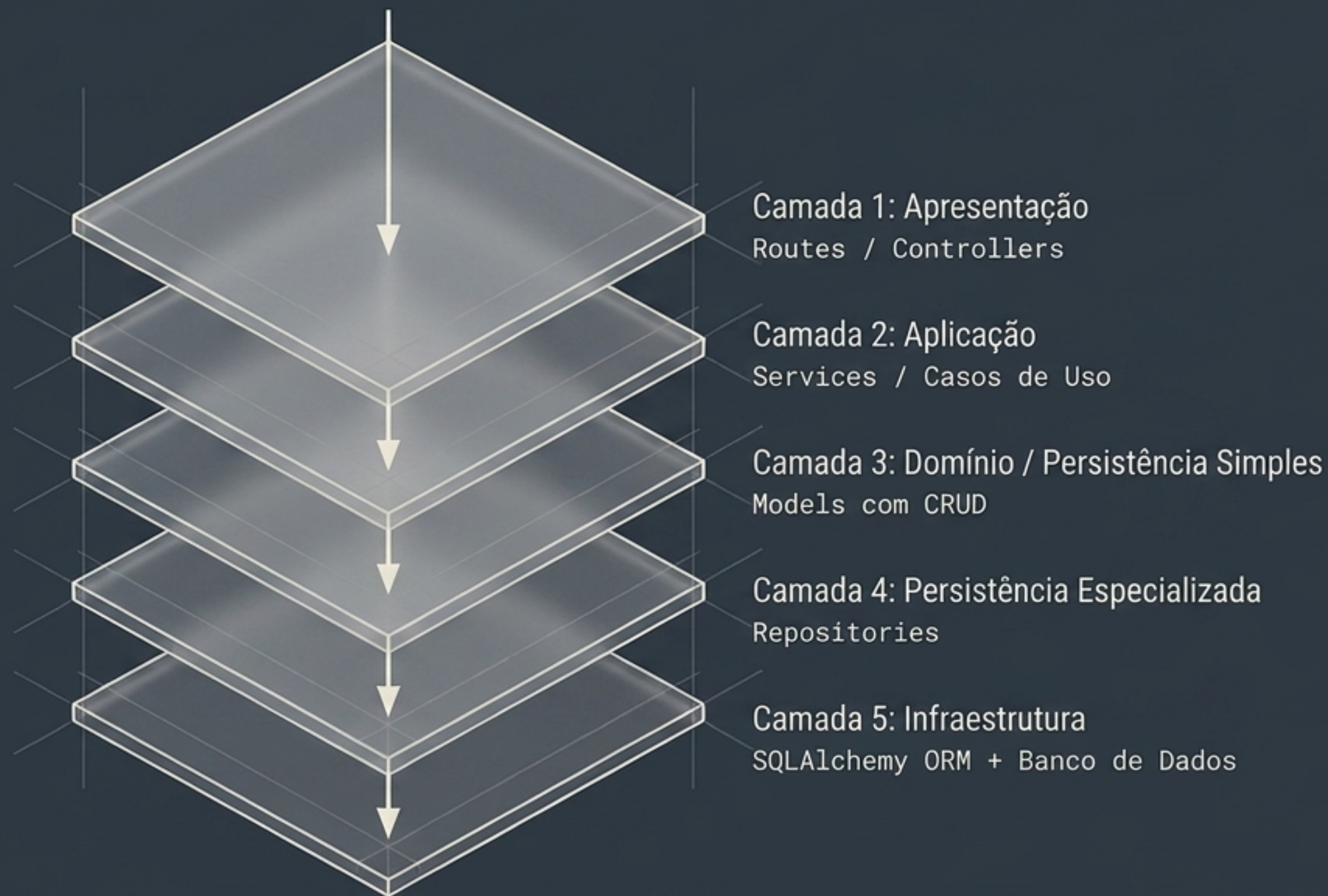
- routes.py
- controllers
- services
- models
- repositories

Módulo Turmas

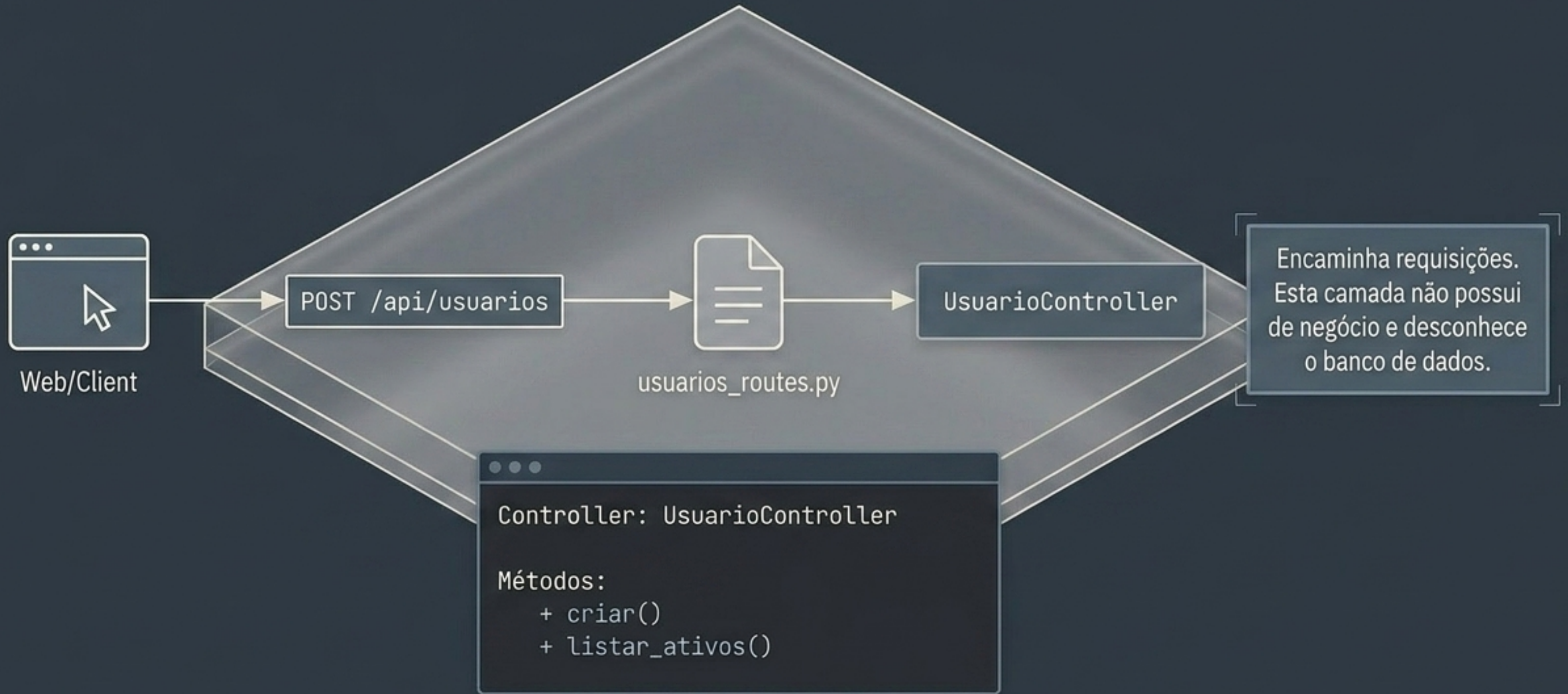
- routes.py
- controllers
- services
- models
- repositories

Cada módulo organiza seu próprio código, mas tudo roda na mesma aplicação e compartilha a mesma infraestrutura.

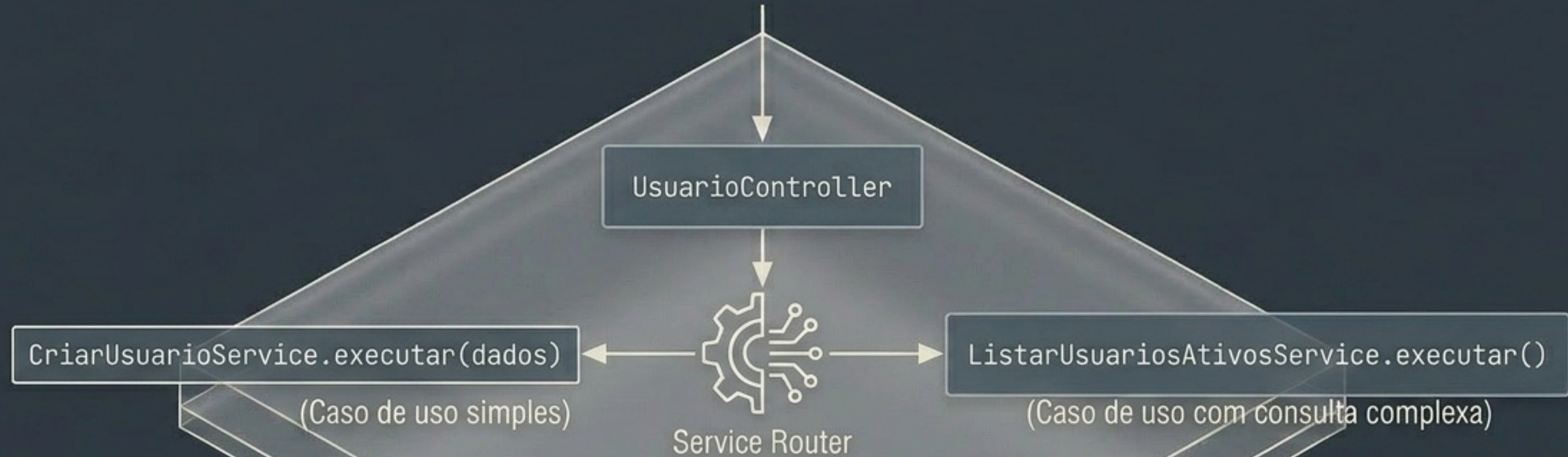
A Anatomia Interna: 5 Camadas de Responsabilidade



A Porta de Entrada: Camada de Apresentação

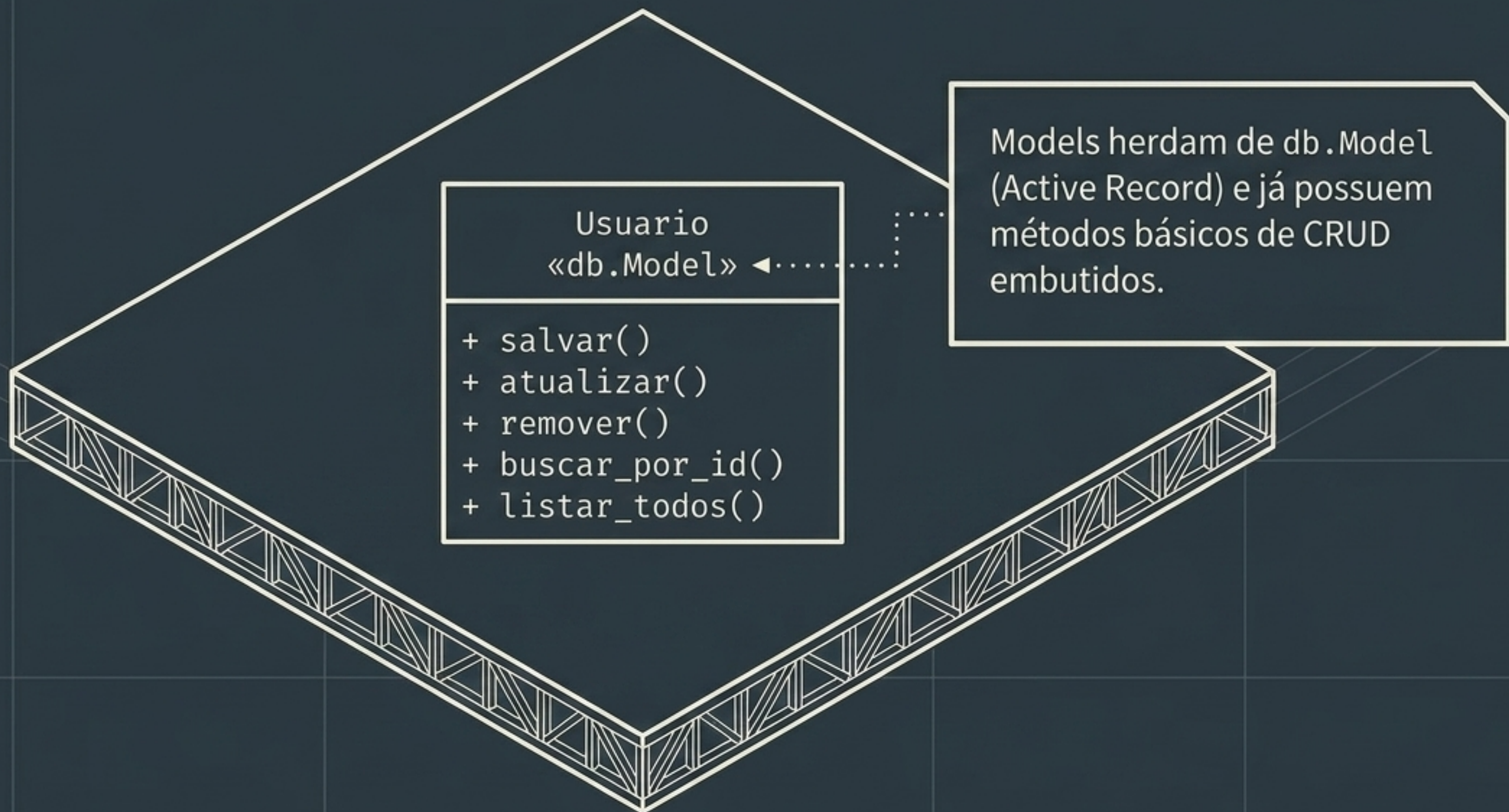


O Maestro do Sistema: Camada de Aplicação

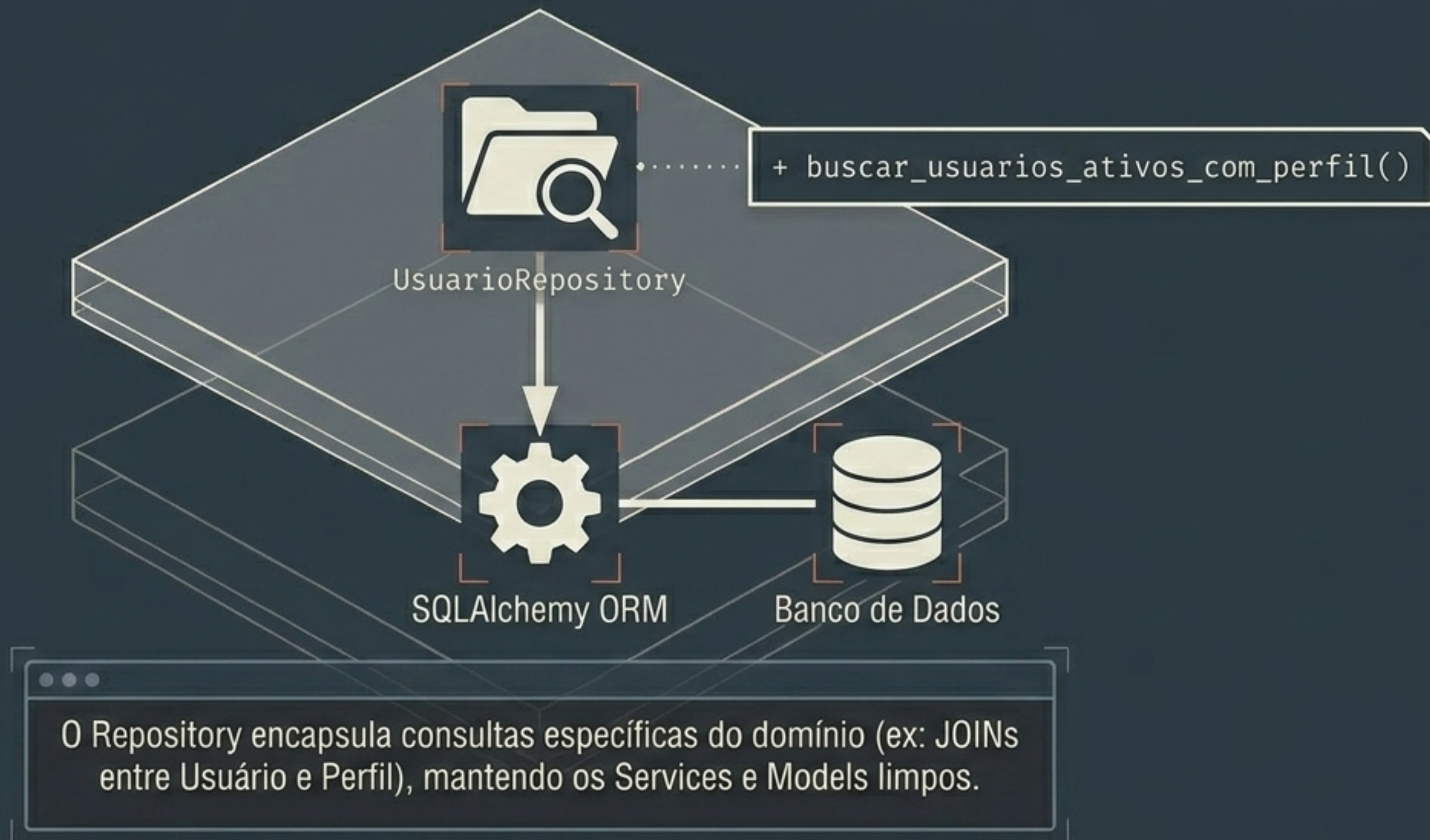


Cada Service representa um único Caso de Uso. Ele orquestra a lógica de negócio e decide como interagir com as camadas inferiores.

Persistência Nativa: Camada de Domínio



O Trabalho Pesado: Repositories e Infraestrutura



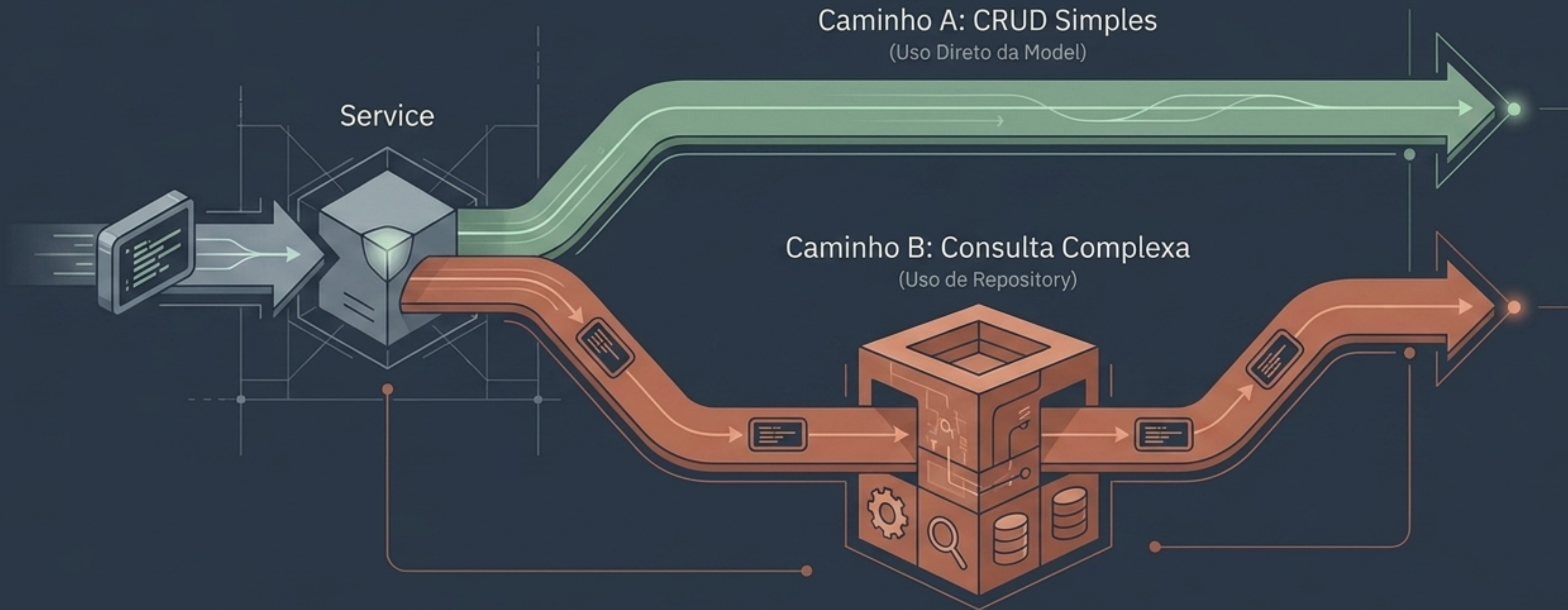
A Arquitetura em Movimento: Dois Caminhos de Execução

A complexidade da operação dita quais camadas serão acionadas.

Caminho A: CRUD Simples
(Uso Direto da Model)

Caminho B: Consulta Complexa
(Uso de Repository)

Service

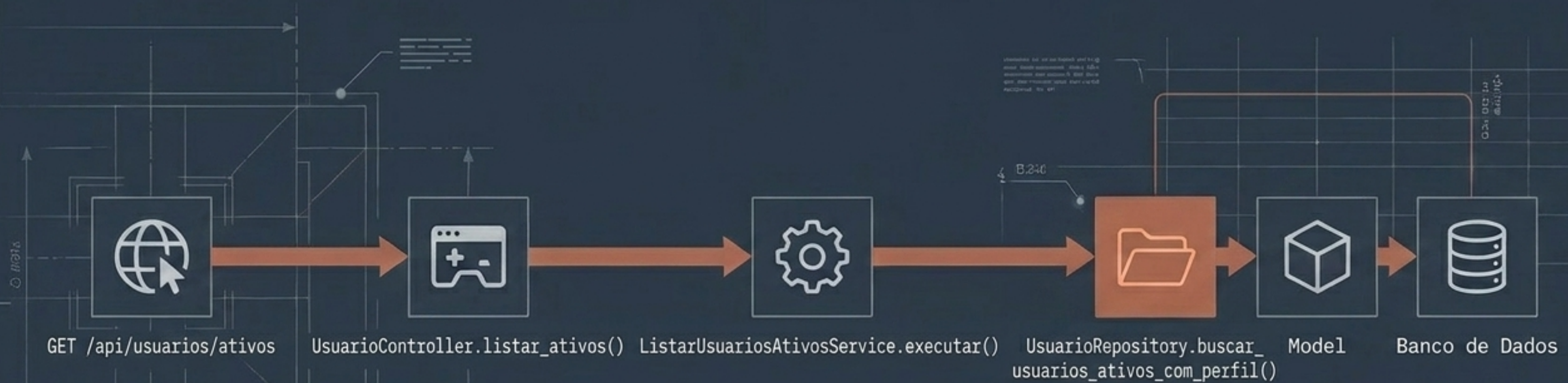


Fluxo A: CRUD Simples via Active Record



Para criar e salvar usuários, o Service usa diretamente a Model, aproveitando a capacidade nativa do ORM atuando como DAO.

Fluxo B: Consultas Complexas via Repository



O Service aciona o Repository para lidar com a lógica de JOINS e filtros avançados, solicitando uma consulta específica antes de persistir os dados.

Matriz de Decisão: Quando Acionar Cada Camada

Cenário	Ação Necessária	Componente Acionado	Justificativa
Operações Básicas	Salvar, Atualizar, Deletar (CRUD)	Direto na `Model`	Evita redundância; o Active Record já resolve o problema.
Recuperação Avançada	Filtros customizados, JOINS, agregações	Uso do `Repository`	Encapsula a complexidade da query (SQLAlchemy) e evita poluir a regra de negócio do Service.

O Guia Prático de Implementação



Endpoints ficam exclusivamente nas rotas e apenas apontam para o Controller.



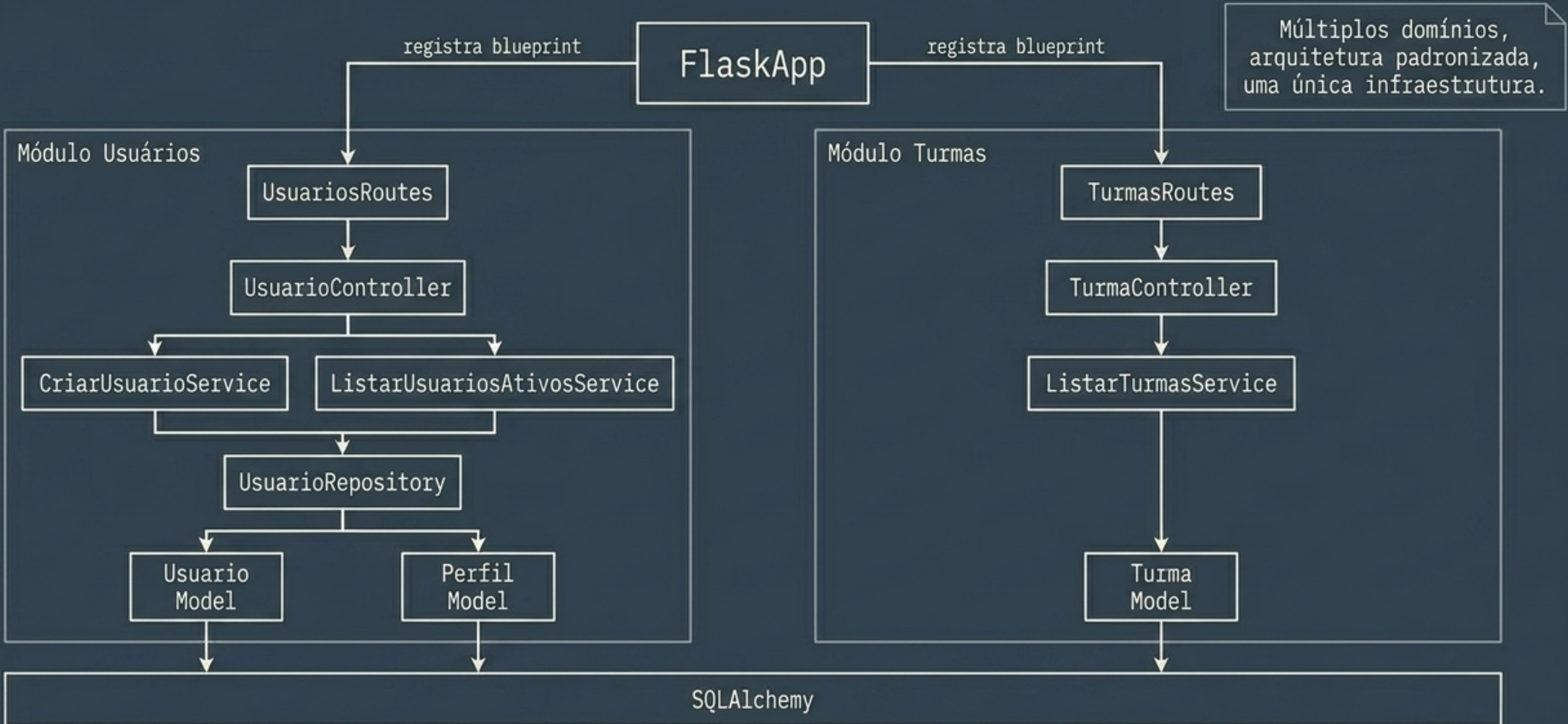
Service simples usa diretamente a Model para persistência.



Service com consulta complexa obrigatoriamente usa Repository.

Arquitetura limpa exige disciplina para não pular etapas ou misturar responsabilidades.

Síntese: 0 Ecossistema Completo em Ação



A Disciplina da Arquitetura Limpa

